



**ROSEBANK
INTERNATIONAL**
University College

RaceDay

Part 1 - Database Design and API Planning

Names:Uhone Sedzani Ravhura

Student No: ST10104495

Module: PROG6212

Table of Contents

Table of Contents	2
1. RaceDay Requirements.....	3
Event Organiser	3
Participant	3
2. Business Rules.....	3
Account and Role Rules.....	3
Event Rules	3
Category Rules	4
Enrolment Rules	4
Result Rules	4
Role-Based Access Rules.....	4
Business Rule Summary	4
Section A - Entity Relationship Diagram (ERD).....	6
Organisers	6
Participants.....	6
Events	6
Categories	7
Enrolments	7
Results	7
Relationships and Cardinalities	8
ERD Diagram	9
Section B - API Endpoint Plan	10
Authentication (Register & Login).....	10
User Profile.....	10
Events	10
Categories	11
Event Enrolments	11
Results	12
Section C - SQL Database Script	13

1. RaceDay Requirements

RaceDay digitises event management for road running, walking, and cycling events in South Africa, replacing paper based registration and disconnected spreadsheets. The system supports exactly two user roles:

Event Organiser

- Create, edit, and delete events.
- Manage event categories.
- Capture participant results.
- View event enrolments.

Participant

- Create an account.
- Browse upcoming events.
- Enter an event by selecting a category.
- View their own enrolments.
- Check their own personal results.

2. Business Rules

Account and Role Rules

1. A Participant must be able to create an account before entering an event.
2. RaceDay must support two main roles: Event Organiser and Participant.
3. An Event Organiser may create events.
4. An Event Organiser may manage the events they are responsible for.
5. A Participant may browse available events.
6. A Participant may view their own enrolments and personal results.
7. Participants must not be allowed to perform Event Organiser functions such as creating or managing events and categories.

Event Rules

8. An event is managed by an Event Organiser.

- 9. An Event Organiser must be able to create an event.
- 10. An Event Organiser must be able to edit an event they manage.
- 11. An Event Organiser must be able to delete an event they manage.
- 12. An event may have one or more categories that Participants can select when entering the event.
- 13. Participants must be able to browse available events.

Category Rules

- 14. An Event Organiser must be able to create categories for an event.
- 15. An Event Organiser must be able to manage categories belonging to an event they manage.
- 16. A category must belong to an event.
- 17. A Participant enters an event by selecting an available category for that event.

Enrolment Rules

- 18. An enrolment records a Participant entering an event.
- 19. Each enrolment must be linked to a Participant.
- 20. Each enrolment must be linked to an event.
- 21. Each enrolment must record the category selected by the Participant.
- 22. A Participant must be able to view their own event enrolments.
- 23. An Event Organiser must be able to view the enrolments for events they manage.
- 24. Participants must not be able to view or manage other Participants' enrolments.

Result Rules

- 25. An Event Organiser must be able to capture participant results.
- 26. A participant result must be associated with the appropriate Participant and event enrolment.
- 27. An Event Organiser must be able to manage results for participants in events they manage.
- 28. A Participant must be able to check their own results.
- 29. Participants must not be able to view or manage another Participant's personal results.

Role-Based Access Rules

- 30. RaceDay must restrict Event Organiser functions to authorised Event Organiser accounts.
- 31. RaceDay must restrict Participant functions to authorised Participant accounts.
- 32. Participants may enter events, view their enrolments and check their own results.
- 33. Event Organisers may create and manage events, manage event categories, capture participant results and view event enrolments.

Business Rule Summary

The RaceDay system uses the Event Organiser and Participant roles to control access to system functionality. Event Organisers are responsible for managing events, categories, enrolments and participant results, while Participants can create accounts, browse events, enter

events by selecting a category, view their own enrolments and check their personal results. These rules are a foundation for the entities, relationships, cardinalities, ERD, API endpoint plan and SQL database.

Section A - Entity Relationship Diagram (ERD)

The RaceDay database consists of six entities, derived directly from the Business Rules above: Organisers, Participants, Events, Categories, Enrolments, and Results. Each entity, its attributes, keys, and relationships are detailed below.

Organisers

Represents the required Admin table. The POE Background's Event Organiser role is preserved as the functional role name; "Admin" is the naming convention requested for the submission structure only.

Attribute	Data Type	PK	FK / Constraint	Explanation
OrganiserId	INT	PK	-	Unique identifier for role-based access.
FullName	NVARCHAR(100)			Identifies the account holder.
Email	NVARCHAR(150)		UNIQUE	Used for login/authentication.
PasswordHash	NVARCHAR(255)			Used for authentication.
PhoneNumber	VARCHAR(20)			
CreatedAt	DATETIME		DEFAULT	

Participants

Represents the required User table. The POE Background's Participant role is preserved as the functional role name; "User" is the naming convention requested for the submission structure only.

Attribute	Data Type	PK	FK / Constraint	Explanation
ParticipantId	INT	PK	-	Unique identifier for role-based access.
FullName	NVARCHAR(100)			Identifies the account holder.
Email	NVARCHAR(150)		UNIQUE	Used for login/authentication.
PasswordHash	NVARCHAR(255)			Used for authentication.
PhoneNumber	VARCHAR(20)			
CreatedAt	DATETIME		DEFAULT	

Events

Attribute	Data Type	PK	FK / Constraint	Explanation
EventId	INT	PK	-	Unique identifier for the event.
OrganiserId	INT		FK -> Organisers.OrganiserId	An event is managed by exactly one Organiser.

Attribute	Data Type	PK	FK / Constraint	Explanation
EventName	NVARCHAR(150)			Needed for Participants to browse events.
EventDate	DATE			A bookable event needs a date.
Location	NVARCHAR(150)			Supports browsing
Status	VARCHAR(20)		DEFAULT	

Categories

Carries a composite UNIQUE constraint on (EventId, CategoryId) so that Enrolments can enforce that a selected category belongs to the same event recorded on the enrolment.

Attribute	Data Type	PK	FK / Constraint	Explanation
CategoryId	INT	PK	UQ (EventId, CategoryId)	Unique identifier for the category.
EventId	INT		FK -> Events.EventId	A category must belong to an event.
CategoryName	NVARCHAR(100)			Participant selects this when entering an event.
DistanceKm	DECIMAL(5,2)			-
EntryFee	DECIMAL(8,2)		DEFAULT	-

Enrolments

Associative entity resolving the Participant-Event-Category relationship. The composite foreign key on (EventId, CategoryId) referencing Categories(EventId, CategoryId) makes it structurally impossible for an enrolment to reference a category from a different event.

Attribute	Data Type	PK	FK / Constraint	Explanation
EnrolmentId	INT	PK	-	Unique identifier for the enrolment.
ParticipantId	INT		FK -> Participants.ParticipantId	Each enrolment belongs to one Participant.
EventId	INT		FK (composite)	Each enrolment is for one Event.
CategoryId	INT		FK (composite) -> Categories(EventId, CategoryId)	Category is validated against the same event.
EnrolmentDate	DATETIME		DEFAULT	-
Status	VARCHAR(20)		DEFAULT	-

Results

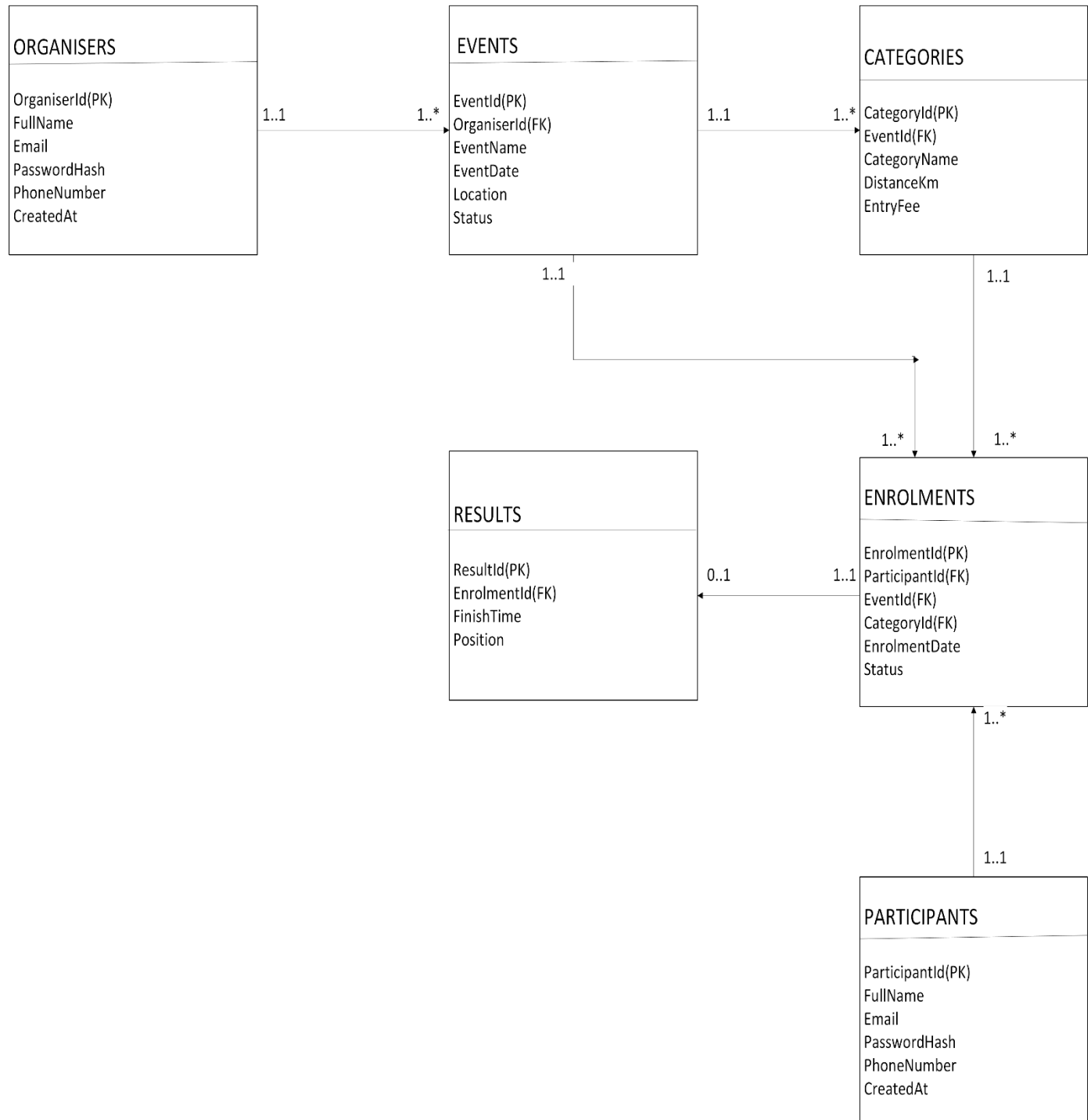
EnrolmentId is NOT NULL and UNIQUE. This enforces that each Result belongs to exactly one Enrolment and that an Enrolment can have at most one Result. An Enrolment may exist without a Result.

Attribute	Data Type	PK	FK / Constraint	Explanation
ResultId	INT	PK	-	Unique identifier for the result.
EnrolmentId	INT		FK -> Enrolments.EnrolmentId, UNIQUE	Links the result to exactly one enrolment.
FinishTime	TIME			[Assumption] Natural minimal field for a race result.
Position	INT			[Assumption] Natural minimal field for a race result.

Relationships and Cardinalities

Entity A	Entity B	Cardinality	Explanation
Organisers	Events	1 -> M	One Organiser manages many Events; each Event has exactly one Organiser.
Events	Categories	1 -> M	One Event has many Categories; each Category belongs to exactly one Event.
Participants	Enrolments	1 -> M	One Participant makes many Enrolments; each Enrolment belongs to one Participant.
Events	Enrolments	1 -> M	One Event receives many Enrolments; each Enrolment is for one Event.
Categories	Enrolments	1 -> M	One Category is selected in many Enrolments, scoped to the same Event.
Enrolments	Results	1 -> 0..1	An Enrolment may or may not yet have a Result.

ERD Diagram



Section B - API Endpoint Plan

The API endpoint plan below covers all six required functional areas: Authentication, User Profile, Events, Categories, Event Enrolments, and Results. It contains 33 endpoints in total, each mapped directly to a business rule. Ownership restrictions are noted in the Role Required column where the user can access only their own data or events they manage.

Authentication (Register & Login)

Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/auth/register/participant	Creates a new Participant account.	Public	FullName, Email, Password	201 Created, new ParticipantId
POST	/api/auth/register/organiser	Creates a new Organiser account.	Public	FullName, Email, Password	201 Created, new OrganiserId
POST	/api/auth/login/participant	Logs in a Participant and returns an access token.	Public	Email, Password	200 OK, auth token
POST	/api/auth/login/organiser	Logs in an Organiser and returns an access token.	Public	Email, Password	200 OK, auth token
POST	/api/auth/refresh-token	Issues a new access token using a valid refresh token.	Participant, Organiser	RefreshToken	200 OK, new token
POST	/api/auth/logout	Logs out the currently authenticated user.	Participant, Organiser	None	200 OK

User Profile

Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/participants/{id}	Retrieves the logged-in Participant's own profile.	Participant	None	200 OK, profile data
PUT	/api/participants/{id}	Updates the logged-in Participant's own profile.	Participant	FullName, PhoneNumber	200 OK, updated profile
GET	/api/organisers/{id}	Retrieves the logged-in Organiser's own profile.	Organiser	None	200 OK, profile data
PUT	/api/organisers/{id}	Updates the logged-in Organiser's own profile.	Organiser	FullName, PhoneNumber	200 OK, updated profile

Events

Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events	Retrieves all events for Participants to browse.	Participant	None	200 OK, list of events
GET	/api/events/{id}	Retrieves the details of a single event.	Participant	None	200 OK, event details
POST	/api/events	Creates a new event owned by the logged-in Organiser.	Organiser	EventName, EventDate, Location	201 Created, new EventId
PUT	/api/events/{id}	Updates an event managed by the logged-in Organiser.	Organiser	EventName, EventDate, Location	200 OK, updated event
DELETE	/api/events/{id}	Deletes an event managed by the logged-in Organiser.	Organiser	None	204 No Content
GET	/api/organisers/{id}/events	Retrieves all events managed by a specific Organiser.	Organiser	None	200 OK, list of events

Categories

Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events/{id}/categories	Retrieves all categories available for a specific event.	Participant	None	200 OK, list of categories
GET	/api/categories/{id}	Retrieves the details of a single category.	Participant	None	200 OK, category details
POST	/api/events/{id}/categories	Creates a new category for an event the Organiser manages.	Organiser	CategoryName, DistanceKm, EntryFee	201 Created, new CategoryId
PUT	/api/categories/{id}	Updates a category belonging to an event the Organiser manages.	Organiser	CategoryName, DistanceKm, EntryFee	200 OK, updated category
DELETE	/api/categories/{id}	Deletes a category belonging to an event the Organiser manages.	Organiser	None	204 No Content

Event Enrolments

Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/enrolments	Enters the logged-in Participant into an event by selecting a category.	Participant	EventId, CategoryId	201 Created, new EnrolmentId
GET	/api/participants/{id}/enrolments	Retrieves the logged-in Participant's own enrolments.	Participant	None	200 OK, list of enrolments

Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/enrolments/{id}	Retrieves the details of a single enrolment.	Participant, Organiser	None	200 OK, enrolment details
DELETE	/api/enrolments/{id}	Cancels the logged-in Participant's own enrolment.	Participant	None	204 No Content
GET	/api/events/{id}/enrolments	Retrieves all enrolments for an event the Organiser manages.	Organiser	None	200 OK, list of enrolments
GET	/api/organisers/{id}/enrolments	Retrieves all enrolments across every event the Organiser manages.	Organiser	None	200 OK, list of enrolments

Results

Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/results	Captures a result for a Participant's enrolment.	Organiser	EnrolmentId, FinishTime, Position	201 Created, new ResultId
PUT	/api/results/{id}	Updates a previously captured result.	Organiser	FinishTime, Position	200 OK, updated result
DELETE	/api/results/{id}	Deletes a captured result.	Organiser	None	204 No Content
GET	/api/results/{id}	Retrieves the details of a single result.	Participant , Organiser	None	200 OK, result details
GET	/api/participants/{id}/results	Retrieves the logged-in Participant's own results.	Participant	None	200 OK, list of results
GET	/api/events/{id}/results	Retrieves all results for an event the Organiser manages.	Organiser	None	200 OK, list of results

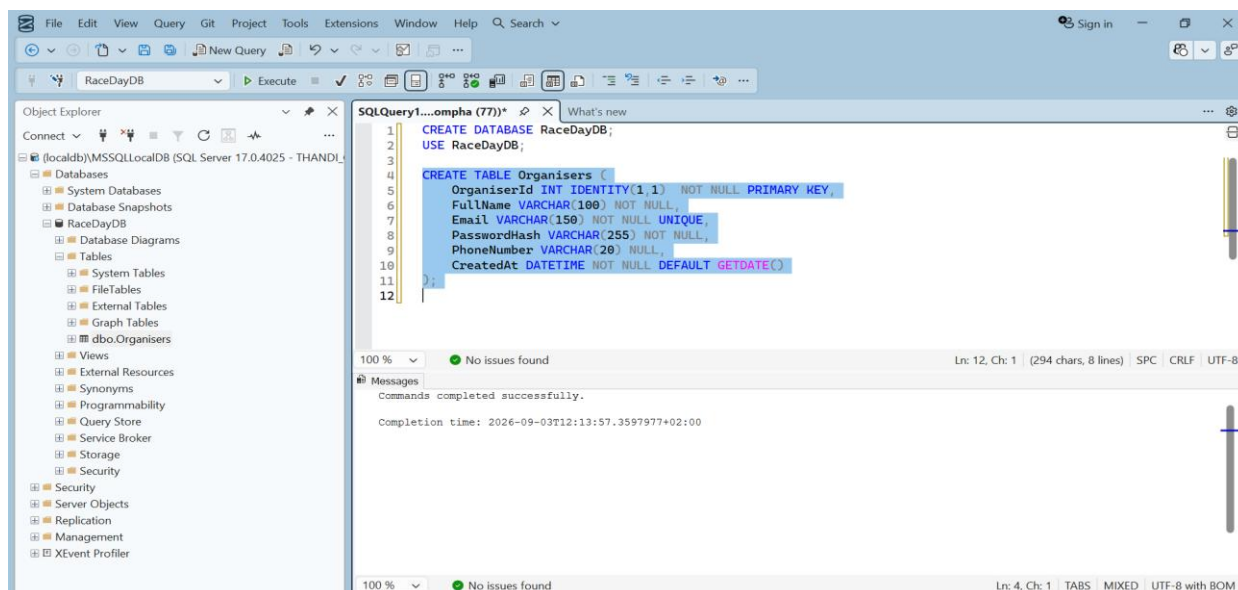
Total endpoints: 33.

Section C - SQL Database Script

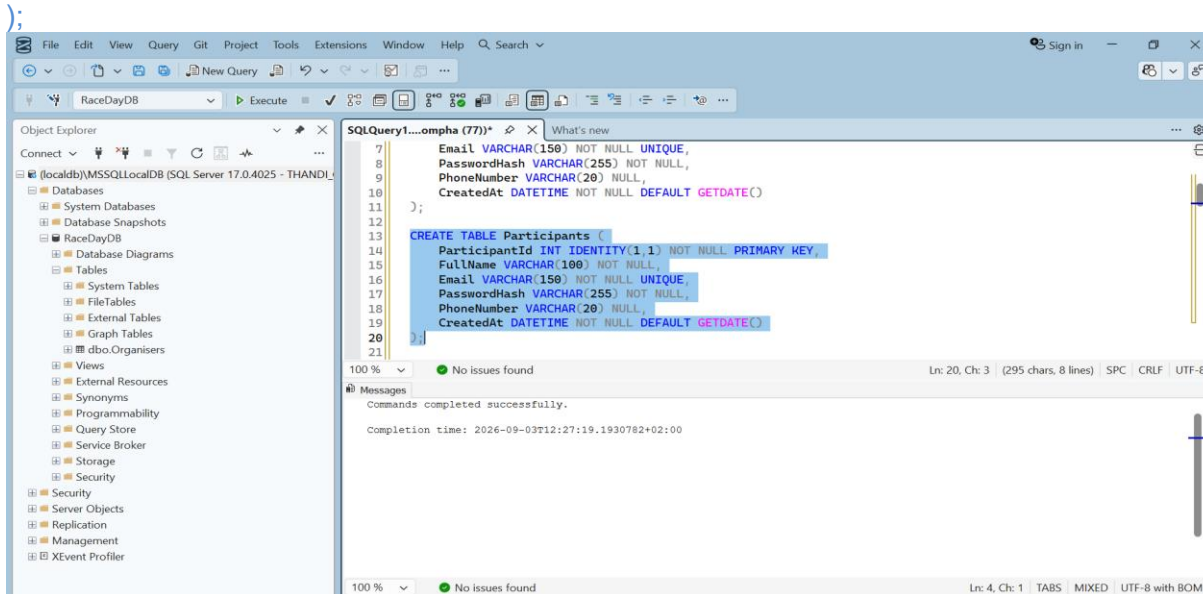
The complete SQL Server script below creates the full database schema matching the ERD exactly, including the composite (EventId, CategoryId) constraint on Categories and Enrolments, the UNIQUE constraint on Results.EnrolmentId, and seed data covering two Organisers, two Participants, three Events, categories for each event, sample enrolments, and results for some enrolments, demonstrating the relationship between Enrolments and Results.

```
CREATE DATABASE RaceDayDB;
USE RaceDayDB;
```

```
CREATE TABLE Organisers (
  OrganiserId INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
  FullName VARCHAR(100) NOT NULL,
  Email VARCHAR(150) NOT NULL UNIQUE,
  PasswordHash VARCHAR(255) NOT NULL,
  PhoneNumber VARCHAR(20) NULL,
  CreatedAt DATETIME NOT NULL DEFAULT GETDATE()
);
```



```
CREATE TABLE Participants (  
    ParticipantId INT IDENTITY(1,1) NOT NULL PRIMARY KEY,  
    FullName VARCHAR(100) NOT NULL,  
    Email VARCHAR(150) NOT NULL UNIQUE,  
    PasswordHash VARCHAR(255) NOT NULL,  
    PhoneNumber VARCHAR(20) NULL,  
    CreatedAt DATETIME NOT NULL DEFAULT GETDATE()  
);
```

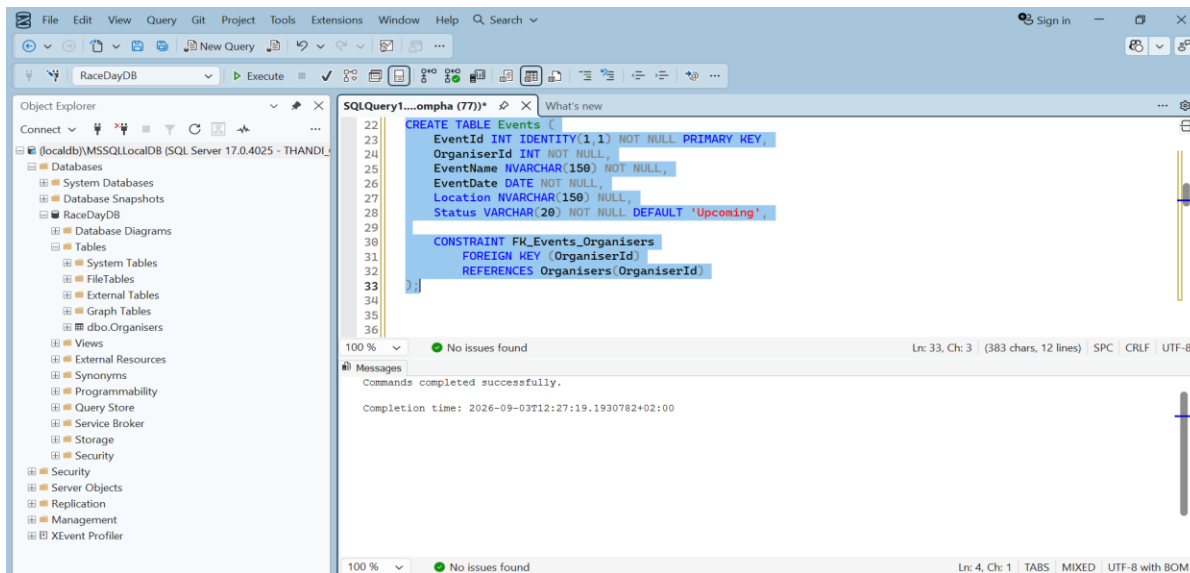


```

CREATE TABLE Events (
    EventId INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    OrganiserId INT NOT NULL,
    EventName NVARCHAR(150) NOT NULL,
    EventDate DATE NOT NULL,
    Location NVARCHAR(150) NULL,
    Status VARCHAR(20) NOT NULL DEFAULT 'Upcoming',

    CONSTRAINT FK_Events_Organisers
    FOREIGN KEY (OrganiserId)
    REFERENCES Organisers(OrganiserId)
);

```



```

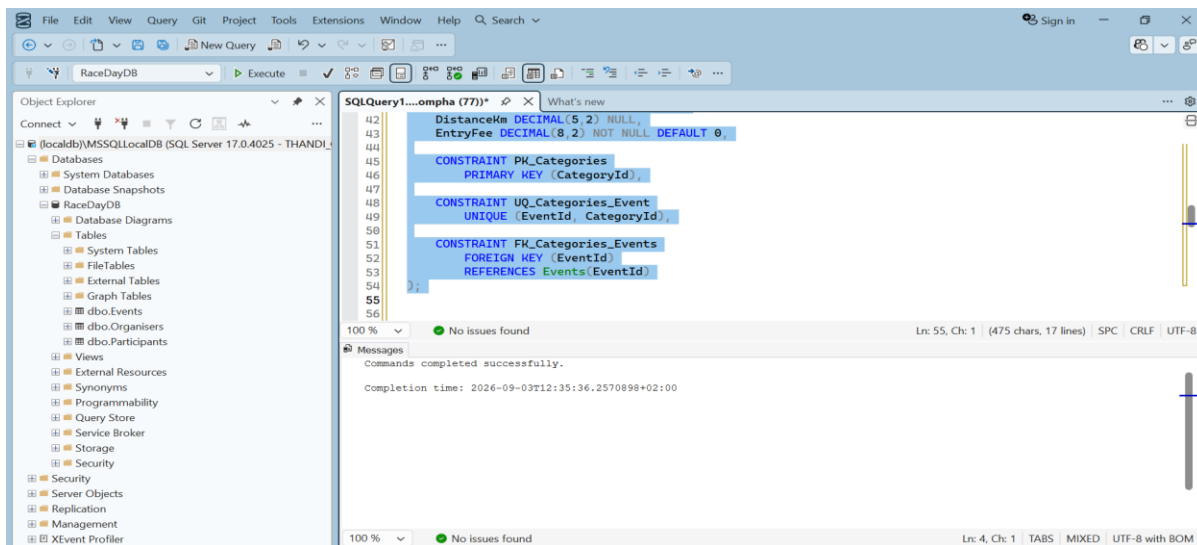
CREATE TABLE Categories (
  CategoryId INT IDENTITY(1,1) NOT NULL,
  EventId INT NOT NULL,
  CategoryName VARCHAR(100) NOT NULL,
  DistanceKm DECIMAL(5,2) NULL,
  EntryFee DECIMAL(8,2) NOT NULL DEFAULT 0,

  CONSTRAINT PK_Categories
    PRIMARY KEY (CategoryId),

  CONSTRAINT UQ_Categories_Event
    UNIQUE (EventId, CategoryId),

  CONSTRAINT FK_Categories_Events
    FOREIGN KEY (EventId)
    REFERENCES Events(EventId)
);

```




```

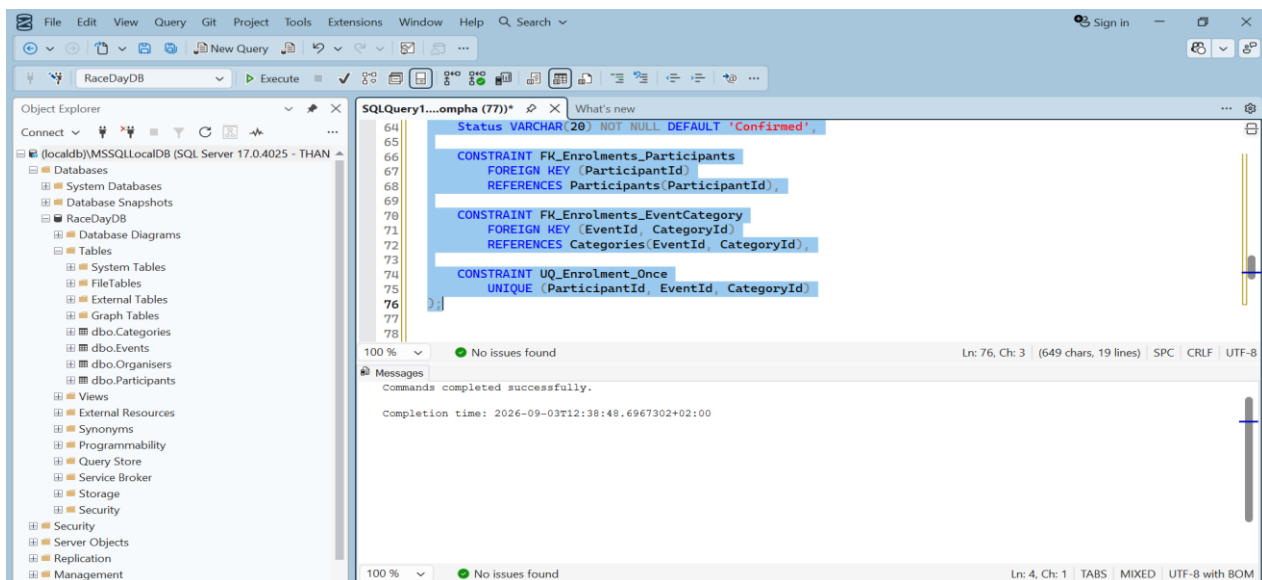
CREATE TABLE Enrolments (
  EnrolmentId INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
  ParticipantId INT NOT NULL,
  EventId INT NOT NULL,
  CategoryId INT NOT NULL,
  EnrolmentDate DATETIME NOT NULL DEFAULT GETDATE(),
  Status VARCHAR(20) NOT NULL DEFAULT 'Confirmed',

  CONSTRAINT FK_Enrolments_Participants
    FOREIGN KEY (ParticipantId)
    REFERENCES Participants(ParticipantId),

  CONSTRAINT FK_Enrolments_EventCategory
    FOREIGN KEY (EventId, CategoryId)
    REFERENCES Categories(EventId, CategoryId),

  CONSTRAINT UQ_Enrolment_Once
    UNIQUE (ParticipantId, EventId, CategoryId)
);

```

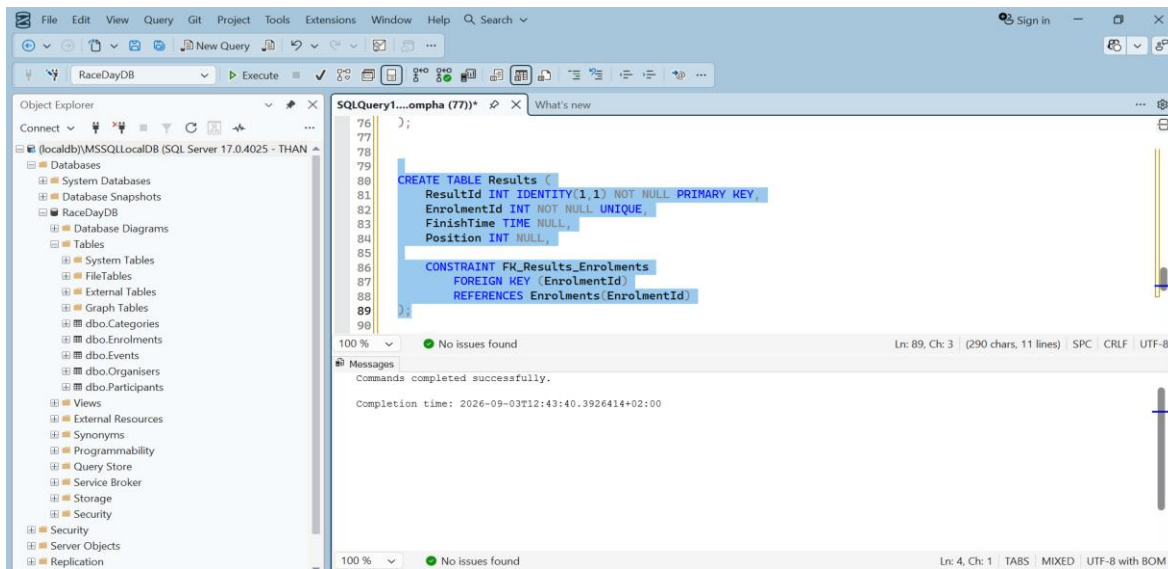


```

CREATE TABLE Results (
    ResultId INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    EnrolmentId INT NOT NULL UNIQUE,
    FinishTime TIME NULL,
    Position INT NULL,

    CONSTRAINT FK_Results_Enrolments
    FOREIGN KEY (EnrolmentId)
    REFERENCES Enrolments(EnrolmentId)
);

```

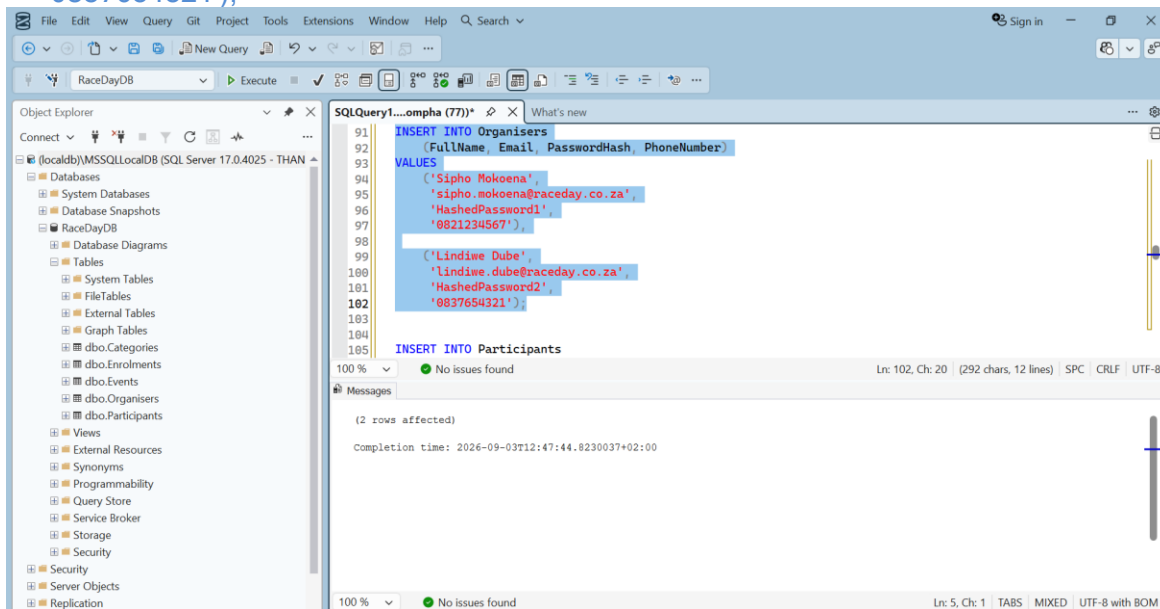


```

INSERT INTO Organisers
(FullName, Email, PasswordHash, PhoneNumber)
VALUES
('Sipho Mokoena',
'sipho.mokoena@raceday.co.za',
'HashedPassword1',
'0821234567'),

('Lindiwe Dube',
'lindiwe.dube@raceday.co.za',
'HashedPassword2',
'0837654321');

```

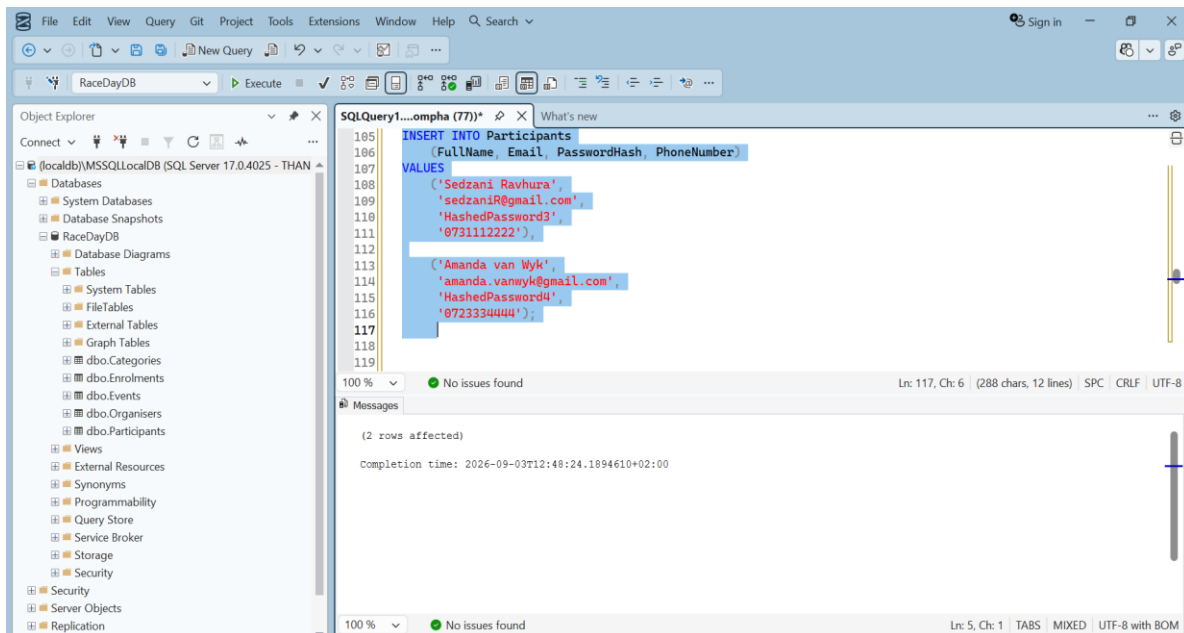


```

INSERT INTO Participants
(FullName, Email, PasswordHash, PhoneNumber)
VALUES
('Sedzani Ravhura',
'sedzaniR@gmail.com',
'HashedPassword3',
'0731112222'),

('Amanda van Wyk',
'amanda.vanwyk@gmail.com',
'HashedPassword4',
'0723334444');

```



```

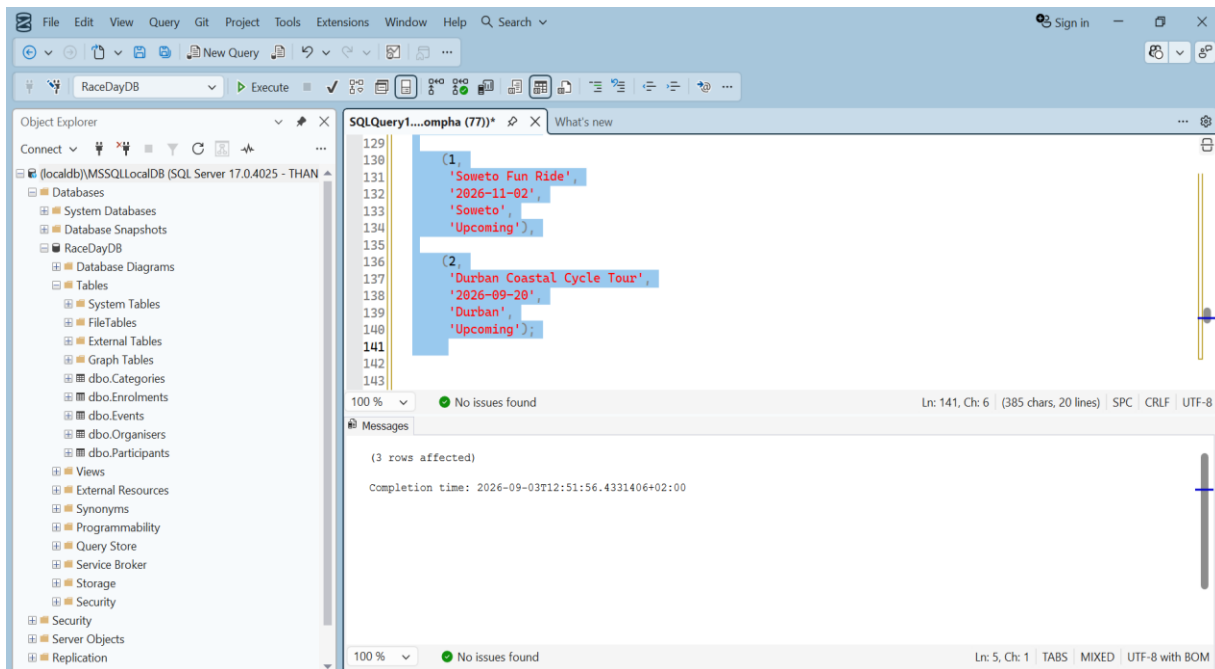
INSERT INTO Events
(OrganiserId, EventName, EventDate, Location, Status)
VALUES
(1,
'Johannesburg City Marathon',
'2026-10-10',
'Johannesburg',
'Upcoming'),

(1,
'Soweto Fun Ride',
'2026-11-02',
'Soweto',
'Upcoming'),

(2,
'Durban Coastal Cycle Tour',
'2026-09-20',
'Durban',

```

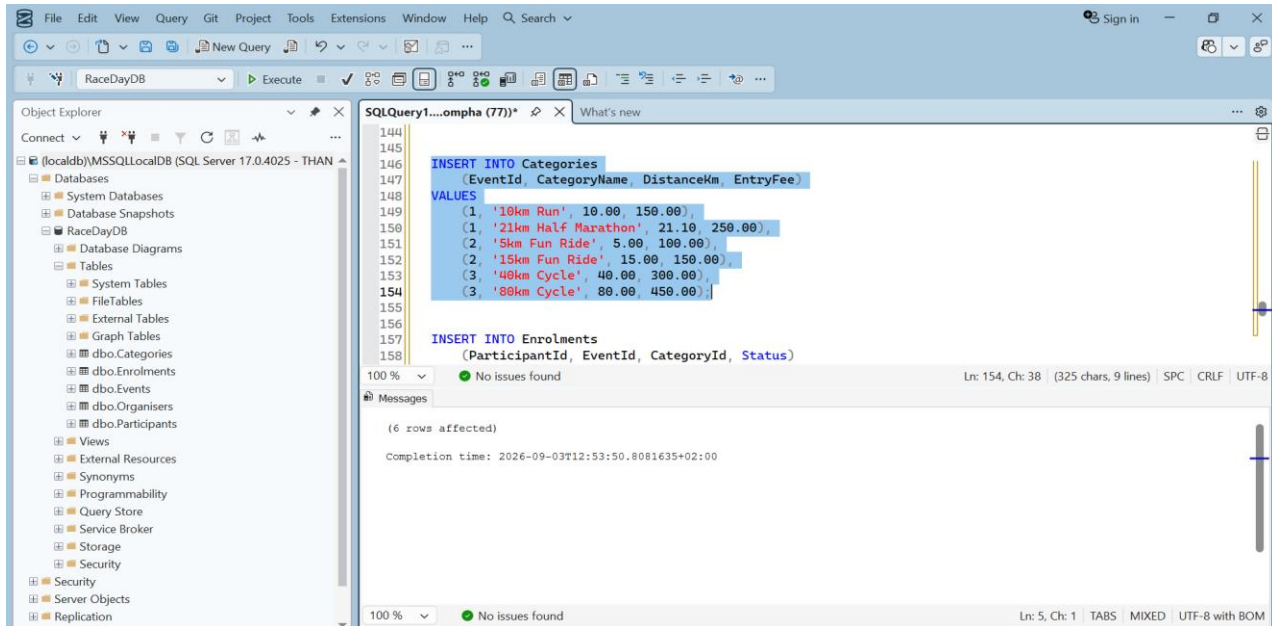
'Upcoming');



INSERT INTO Categories
(EventId, CategoryName, DistanceKm, EntryFee)
VALUES

```

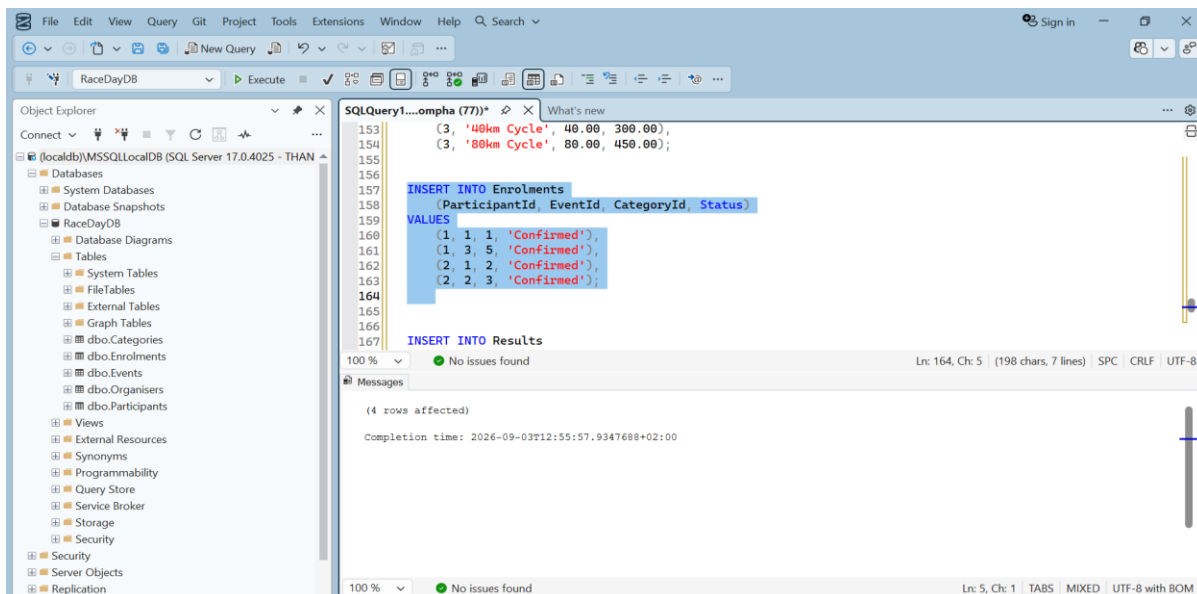
(1, '10km Run', 10.00, 150.00),
(1, '21km Half Marathon', 21.10, 250.00),
(2, '5km Fun Ride', 5.00, 100.00),
(2, '15km Fun Ride', 15.00, 150.00),
(3, '40km Cycle', 40.00, 300.00),
(3, '80km Cycle', 80.00, 450.00);
  
```



```

INSERT INTO Enrolments
(ParticipantId, EventId, CategoryId, Status)
VALUES
(1, 1, 1, 'Confirmed'),
(1, 3, 5, 'Confirmed'),
(2, 1, 2, 'Confirmed'),
(2, 2, 3, 'Confirmed');

```

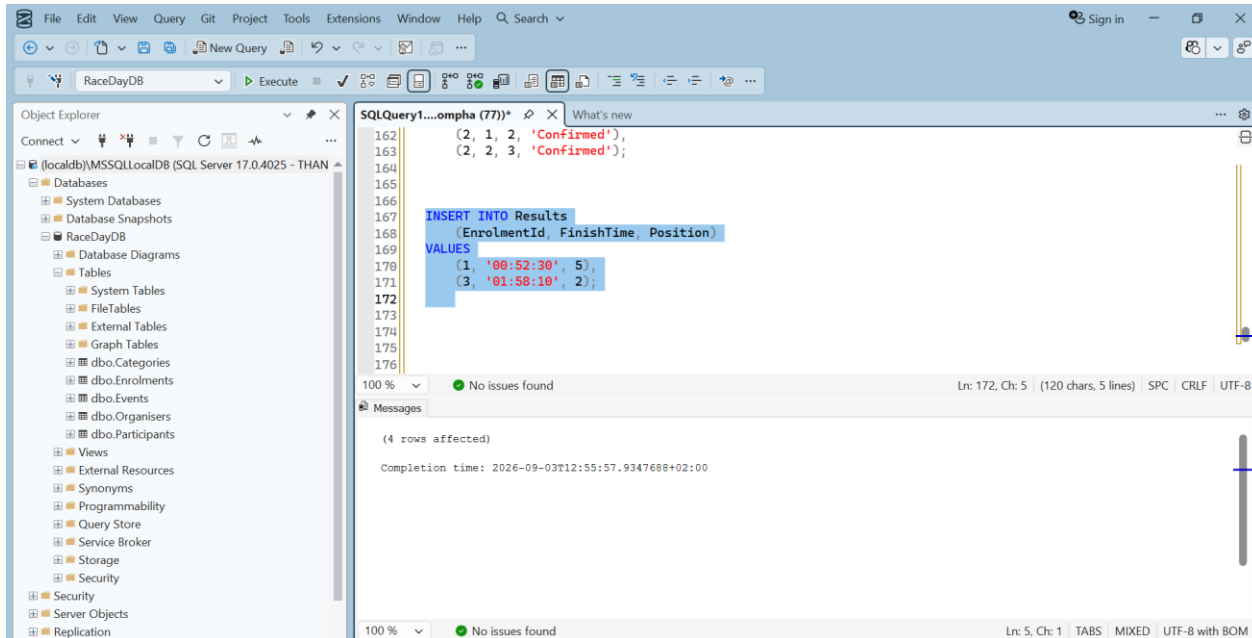


```

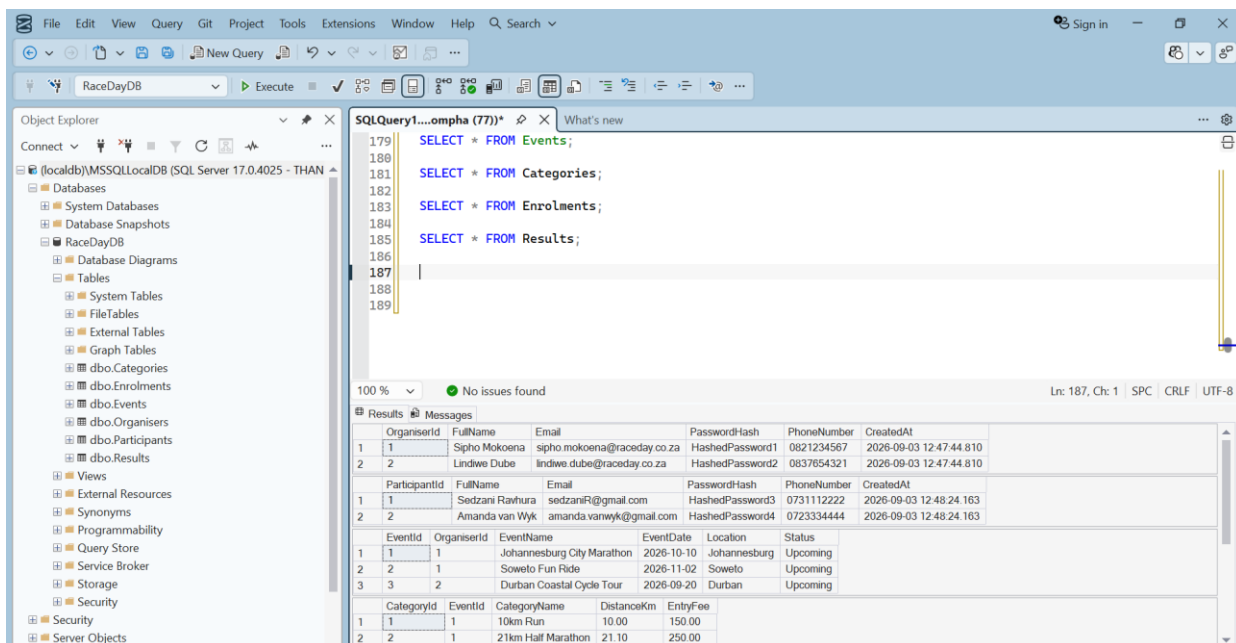
INSERT INTO Results

```

(EnrolmentId, FinishTime, Position)
VALUES
(1, '00:52:30', 5),
(3, '01:58:10', 2);



Tables populated



The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure for 'RaceDayDB', including tables like 'dbo.Enrolments', 'dbo.Events', 'dbo.Participants', and 'dbo.Results'. The central query window, titled 'SQLQuery1....ompha (77)*', contains a SQL query with line numbers 173 through 187. The query is a SELECT statement with multiple FROM clauses, each preceded by a comment: 'SELECT * FROM Organisers;', 'SELECT * FROM Participants;', 'SELECT * FROM Events;', 'SELECT * FROM Categories;', 'SELECT * FROM Enrolments;', and 'SELECT * FROM Results;'. The status bar indicates 'No issues found' and 'Ln: 185, Ch: 23 | (165 chars, 11 lines) | SPC | CRLF | UTF-8'. Below the query, the 'Results' pane displays two tables of data.

173
174
175
176
177
178
179
180
181
182
183
184
185
186
187

```
SELECT * FROM Organisers;  
SELECT * FROM Participants;  
SELECT * FROM Events;  
SELECT * FROM Categories;  
SELECT * FROM Enrolments;  
SELECT * FROM Results;
```

100 % No issues found Ln: 185, Ch: 23 | (165 chars, 11 lines) | SPC | CRLF | UTF-8

Results Messages

2	2	1	21km Half Marathon	21.10	250.00	
3	3	2	5km Fun Ride	5.00	100.00	
4	4	2	15km Fun Ride	15.00	150.00	
5	5	3	40km Cycle	40.00	300.00	
6	6	3	80km Cycle	80.00	450.00	

EnrolmentId	ParticipantId	EventId	CategoryId	EnrolmentDate	Status
1	1	1	1	2026-09-03 12:55:57.930	Confirmed
2	2	1	3	2026-09-03 12:55:57.930	Confirmed
3	3	2	1	2026-09-03 12:55:57.930	Confirmed
4	4	2	2	2026-09-03 12:55:57.930	Confirmed

ResultId	EnrolmentId	FinishTime	Position
1	1	00:52:30.0000000	5
2	2	01:58:10.0000000	2

